scientific reports

OPEN

Intelligent resource allocation in internet of things using random forest and clustering techniques

Nahideh Derakhshanfard[1]✉, Lida Hosseinzadeh[1], Fahimeh Rashid Jafari[1] & Ali Ghaffari[1,2,3]

The Internet of Things has proliferated, and the number of devices integrated into intelligent networks has made resource management and allocation one of the most critical challenges. The intrinsic constraints of IoT devices, such as energy consumption, limited bandwidth, and reduced computational power, have increased the demand for more intelligent and efficient resource allocation strategies. Numerous current resource allocation methods, such as evolutionary algorithms and multi-agent reinforcement learning, are grossly inefficient at adapting well to IoT networks in light of dynamic and rapid changes due to the inherent computational complexity and high cost. This paper proposes an intelligent resource allocation approach for Internet of Things (IoT) networks that integrates clustering and machine learning techniques. Initially, IoT devices are grouped using the K-Means algorithm based on features such as energy consumption and bandwidth requirements. A Random Forest model is then trained to accurately predict the resource needs of each cluster, enabling optimal allocation. Simulation results show that the proposed approach improves prediction accuracy to 94%, reduces energy consumption by 20%, and decreases response time by 10% compared to existing methods. These results highlight the effectiveness of the approach in managing resources in dynamic and scalable IoT environments.

Keywords Internet of things, Resource management, Resource allocation, Random forest, K-means clustering, Dynamic resource scheduling

With the rapid expansion of digital technologies, IoT has become one of the key elements transforming modern life. It interconnects billions of devices, enabling smooth data exchange and the creation of smart networks. However, such rapid growth has also brought several challenges in resource management, energy optimization, and quality of service. In this context, inventive methods of resource optimization and effective data management have gained high attention. The rapid development of these devices generating vast volumes of data while requiring relatively constrained resources in terms of energy, bandwidth, and processing capabilities has made fog and edge computing environments imperative for efficient resource management[1–3].

Besides, data protection has become a concern at all levels[4]. Traditional resource allocation methods usually aim at the minimization of cost or execution time and do not take into account multiple objectives at the same time, such as energy consumption reduction[5,6], service quality assurance, and adaptation to dynamic condition changes[1]. Further, due to dynamic changes in the needs of IoT devices and the limitations of resource allocation[7,8] in dynamic environments, sudden changes in user resource requirements make resource allocation even more complex[6,9]. The high energy consumption of devices and the need to reduce the energy consumption of IoT devices is one of the most critical problems in prolonging the device's lifespan[5,9]. Delay management: Delay reduction is a critical issue for real-time applications in intelligent transportation[10]. Although some research works on resource allocation in IoT, the necessity of integrating advanced techniques such as deep learning, metaheuristic optimization, and fault tolerance has been explored to a lesser extent. Equally important is the requirement for integrated approaches aimed at simultaneously reducing delay and energy costs and improving fault tolerance[11–13]. The purpose of this paper is to dynamically enhance resource allocation using online learning methodologies[1]. The methodology presented here proposes a hybrid approach in which machine learning—more precisely, Random Forest—and clustering techniques are combined to allocate resources. First, IoT devices are clustered according to standard features to handle similar needs in different classes efficiently.

Then, the Random Forest model is applied to predict future demands and allocate optimal resources. This scheme allows for energy efficiency improvement, delay reduction, and better network performance overall[14],

[1]Department of Computer Engineering, Ta.C, Islamic Azad University, Tabriz, Iran. [2]Department of Computer Engineering, Faculty of Engineering and Natural Science, Istinye University, Istanbul, Turkey. [3]Department of Computer Engineering, Khazar University, Baku, Azerbaijan. ✉email: n.derakhshan@iaut.ac.ir

and it also enables efficient resource allocation between users[5,6]. Hence, it proposes combining the Random Forest model and K-Means clustering for resource allocation in IoT networks. First, IoT devices are clustered by the K-Means clustering algorithm depending on energy consumption and bandwidth demands. Afterward, the Random Forest model is used to predict the resource needs for every cluster. This method, which is of high accuracy, identifies complex patterns in the use of resources and allocates resources effectively based on the actual needs of the devices. The proposed methodology also shows the ability to adapt in real time to environmental changes and aims to maximize efficiency while reducing energy consumption[1,14].

Generally, the method proposed in this paper can solve the significant resource allocation challenges in IoT using machine learning and clustering techniques, which can significantly improve IoT networks' efficiency and energy consumption. In this paper, a high accuracy in resource needs prediction was achieved by combining the Random Forest model with K-Means clustering. The method allows devices to receive more optimal resources, thus increasing system efficiency. By allocating resources based on actual needs, this scheme achieves the energy efficiency of devices and potentially leads to an extension in their lifetime. This factor is significant in reducing costs and maintaining IoT networks. The use of K-Means clustering helps to handle resources for many devices, and by simplifying management issues, it enables IoT networks to perform better and solve more critical problems. Simulation results have shown that the proposed solution is much better at reducing energy consumption, improving quality-of-service assurance, and reducing delay than traditional methods. This goal's achievement confirms the proposed method's high potential value in the practical application in the dynamic and complex IoT environment.The main focuses of the paper are as follows:

- A hybrid framework was proposed for resource allocation in IoT networks using K-Means clustering and Random Forest prediction.
- Device-level features such as energy consumption and bandwidth requirements were considered in both clustering and prediction processes.
- The algorithmic structure of the proposed model was elaborated through a step-by-step analysis of clustering and decision-making based on Random Forest.
- A mathematical formulation of the resource allocation process was provided, incorporating both cluster-level and device-level features to enhance analytical clarity and generalizability.

The rest of the paper is organized as follows: "Related works" describes related works, "Proposed method" introduces the proposed method, "Evaluation of the proposed algorithm" presents the simulation results, and the final section is dedicated to the proposed method's conclusion.

Related works

Limiting the resource availability in IoT networks, for instance, energy, bandwidth, and computation power, optimizes resource allocation. Many researchers have thus put forth various methods for improving such networks' resource management and allocation mechanisms. In[14,15], some researchers, based on some research work focusing on the application of edge and fog computing, reduce latency further to increase efficiency in IoT networks. This approach relocates part of the processing to locations closer to the data source, reducing bandwidth consumption and transmission delay. For instance, in one study, researchers employed fog computing to manage resources in intelligent transportation, which reduced latency and further improved system efficiency[14].

Evolutionary algorithm-based methods have also been widely used for resource allocation in IoT networks. Bat and whale optimization have been applied to solve resource allocation problems in IoT's dynamic and complex environments. Although advantageous in resource allocation optimization, these methods generally require high computational resources and long computation times, which is unrealistic for most IoT applications because of resource limitations. Conversely, reinforcement learning and multi-agent approaches have also been proposed for resource management in IoT. Examples include multi-agent reinforcement learning-based methods[16] proposed for different scenarios of resource allocation in IoT to enhance efficiency and resource management. However, these approaches require further improvement since they often present profound computational complexities and incur high costs; hence, they are not optimal for all cases.

The remaining parts of the literature were related to machine learning models for resource allocation in IoTs; classification algorithms like Random Forest help identify resource requirements and thereby apply the intelligent resource allocation selection policy. Examples include using the Random Forest model to make predictions about the energy consumption pattern of IoT devices in order to plan accordingly for resource allocation. Although many papers have been published regarding resource allocation in IoT, there are still some gaps in this area. Most of the developed methods are concerned with the quality of service or reduction of delay while giving less consideration to energy optimization as an important factor.Most of the listed algorithms are single-point or device-based resource allocation methods. There is still a lack of scalable and holistic methods for performing resource management when facing many IoT devices. Herein, we propose integrating Random Forest and K-Means clustering algorithms as a new IoT network resource allocation approach. The method presented will enable the classification of similar devices using the K-Means algorithm and the Random Forest model for predicting resource needs with high accuracy to achieve optimality and scalability in resource allocation, contributing to enhanced efficiency, reduced energy consumption, and adaptation to dynamic changes[1]. Some of these methods are presented in Table 1.

Proposed method

The following paper, therefore, proposes an intelligent technique for resource allocation in IoT networks using machine learning algorithms and clustering techniques. The proposed approach will contribute to improving efficiency, reducing energy consumption, and optimizing resources by IoT devices. The proposed system uses

References	Paper title	Objectives	Advantages	Disadvantages
17	Efficient Resource Allocation and QoS Enhancements of IoT with Fog Network	Improve resource allocation and QoS in IoT networks using fog computing	Reduce network delay and enhance QoS.	Limited to fog scenarios and scalability issues
18	Energy Efficiency Maximization for Active RIS-Aided Integrated Sensing and Communication	Maximize energy efficiency in sensing and communication with RIS technology.	Increase energy efficiency in communication tasks.	Complex implementation is limited to RIS systems.
19	Energy Efficient Resource Allocation Algorithm for Agriculture IoT	Develop an energy-efficient resource allocation algorithm for agricultural IoT	Reduce energy consumption specific to agricultural applications.	Limited to agriculture with low generalizability potential
20	IoT Resource Allocation and Optimization Based on Heuristic Algorithm	Optimize IoT resource allocation using heuristic algorithms	Simple and effective in resource management	Less flexibility in dynamic IoT environments
21	A Deep Learning Approach for IoT Traffic Multi-Classification in a Smart-City Scenario	Improve accuracy and efficiency in IoT traffic classification using deep learning	High accuracy (up to 99.9%) in classification, suitable for real-world scenarios	High computational cost for processing extensive data
5	A Game-Theoretic Approach for Increasing Resource Utilization in Edge Computing Enabled IoT	Improve resource utilization and allocation using game theory in edge computing for IoT	Distributed solutions with game theory for resource optimization and cost reduction	High computational complexity and need for multiple simulations
22	Machine learning-based demand response scheme for IoT-enabled PV	Develop a short-term prediction model for energy consumption and renewable energy generation using machine learning	Reduce electricity cost and peak demand using dynamic pricing and demand response systems.	Requires complex system implementation and high integration costs for renewable energy
23	MEC Enabled Cooperative Sensing and Resource Allocation for Industrial IoT Systems	Optimize resource allocation and cooperative sensing in industrial IoT systems	Reduce estimation errors and delay with optimized resource allocation	High algorithmic complexity and need for high computational power
24	Resource allocation for content distribution in IoT edge cloud computing environments using deep reinforcement learning	Improve content distribution efficiency and resource allocation in edge and cloud environments using deep reinforcement learning	Reduce network delay and improve resource allocation efficiency	Dependence on high computational resources and network complexity
25	Network Resource Allocation Method based on IoT	Provide a distributed resource allocation method based on blockchain and federated learning for improved network security and efficiency	Increase network security and scalability using a dual blockchain structure	Blockchain implementation complexity and energy consumption issues
26	Optimal Edge Resource Allocation in IoT-Based Smart Cities	Optimize edge resource allocation in smart city IoT systems to reduce service response time	Reduce service response time and improve QoS in smart city IoT systems.	Poor adaptation to dynamic changes in service demand
27	Resource Allocation Based on Predictive Load Balancing Approach in Multi-Cloud Environment	Improve load balancing and resource allocation in multi-cloud environments using adaptive prediction	Improve load balancing and resource allocation in multi-cloud environments	Limited by multi-cloud environments
28	Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments	Improve resource allocation, reduce delay, and energy consumption in distributed AI tasks using deep reinforcement learning	Reduce system costs and improve energy efficiency with a two-phase reinforcement learning model	High learning complexity and dependence on strong computational resources
13	Resource Utilization Based on Hybrid WOA-LOA Optimization with Credit-Based Resource Aware Load Balancing and Scheduling Algorithm for Cloud Computing	Reduce communication costs and improve processing time in IoT resource allocation using EHO optimization	Faster convergence time and reduced communication cost in IoT networks	Limited compatibility with dynamic environments and device diversity
11	Optimal Energy-efficient Resource Allocation and Fault Tolerance scheme for task offloading in IoT-Fog Computing Networks	Improve energy-efficient resource allocation and fault tolerance for task offloading in IoT-Fog networks	Improve fault tolerance and reduce delay in IoT networks	Complexity in implementation and need for advanced resources to manage faults

Table 1. Summary of related works.

the latest machine learning techniques coupled with clustering techniques to efficiently manage resources in these IoT networks. It solves most of the complexities related to large-scale dynamic networks and provides an invaluable contribution to reducing energy consumption and enhancing the performance of IoT networks. The proposed architecture is based on a few steps forged for this particular resource allocation activity in dynamic and complex IoT environments. Random Forest resource allocation can be used to develop the efficiency of resource distribution in an IoT network. Also, clustering enables better pattern identification in data by aggregating devices or nodes based on their common characteristics, such as energy consumption, request type, or communication quality. The combination of both eventually leads to wiser resource allocation and network optimization. Figure 1 illustrates proposed method's workflow.

Mathematical formulation

The goal is to optimize resource distribution by minimizing energy consumption and response time while maximizing prediction accuracy. Each IoT device is first described by a set of features, including energy consumption, data volume, network quality (such as bandwidth and latency), request priority, and the device's maximum tolerable delay. These features form a feature vector $\mathbf{x}_i$ for each device, which serves as input to the K-Means algorithm. This clustering algorithm groups devices with similar behavioral patterns into clusters, thereby reducing data complexity and facilitating moreeffective resource prediction.

After clustering, a Random Forest model is trained using both device-level and cluster-level features to predict the required resources for each device. These resources may include CPU power, memory, bandwidth, or energy. The trained model generates predictions $\hat{y}_i$ based on the input features of each device. The overall objective is

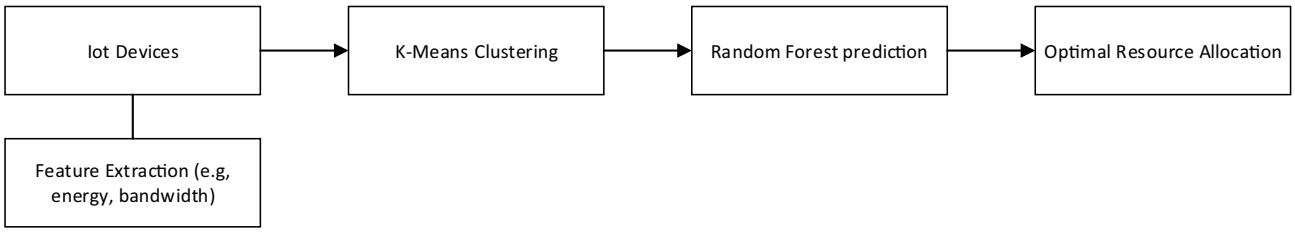

Fig. 1. Workflow of proposed method.

to allocate these resources in a way that optimally balances three goals: minimizing total energy consumption across devices, reducing total system response time, and maximizing the accuracy of the model's predictions. The resource allocation process is subject to several constraints, including limited resources available to each cluster, device-specific maximum energy consumption limits, and quality-of-service (QoS) requirements such as latency thresholds. Each IoT device is represented by the feature vector:

$\mathbf{x}_i = (E_i, D_i, Q_i, P_i, L_i)$

where E_i is the energy consumption, D_i is the data volume, Q_i is the network quality, P_i is the request priority, and L_i is the maximum tolerable delay for device. K-Means clustering minimizes the total within-cluster variance:

$$\min \sum_{i=1}^{N} \left\| \mathbf{x}_i - \mathbf{C}_{k(i)} \right\|^2$$

where $\mathbf{C}_{k(i)}$ is the center of the cluster to which device *i* is assigned. The Random Forest model is then used to predict the required resources and The overall objective function is defined as.

$$\hat{y}_i = f\left(\mathbf{x}_i\right)$$

$$\min \left(\alpha \sum_{i=1}^{N} E_i + \beta \sum_{i=1}^{N} T_i - \gamma \cdot \mathrm{ACC} \right)$$

where T_i is the response time for device *i*, ACC is the prediction accuracy of the Random Forest model, and α, β, γ are weighting coefficients that control the trade-offs among objectives.This optimization is subject to the following constraints, Cluster-level resource limits Equation, QoS latency constraint for each device as Equation, Maximum energy consumption per device as Equation .

$$\sum_{i \in C_k} \hat{y}_i \leqslant R_k \forall k \in \{1, ..., K\}$$

$$T_i \leqslant L_i \forall i$$

$$E_i \leqslant E_{\max} \forall i$$

Stages of resource allocation using random forest in IoT

low are the fundamental resource allocation steps for IoT networks using the Random Forest algorithm.

- Identification and Characterization of Attributes: The first step in resource optimization in IoT networks is identifying features that may impact resource allocation. These features typically contain information related to device status and network requirements. The main features often include device energy consumption, request priorities, data transmission volume, processing usage, and network conditions.
- Data Collection and Preparation: The Random Forest model needs historical data for training, which includes feature engineering like collecting data in the past and labeling. With properly prepared datasets, this model will learn the pattern and make the best predictions possible on resource allocations.
- Training the Random Forest Model: The prepared data is used to train the Random Forest model at this stage. The Random Forest algorithm will build multiple decision trees that cooperatively decide on resource allocation by considering various conditions. Each decision tree can focus on specific resource allocations, including bandwidth, processing power, or energy distribution. In turn, the Random Forest learns how to adapt resource allocation based on changes in input features, such as device and network status. Using metrics such as feature importance allows the model to identify which types of features—energy consumption in this case—will have the most value in resource prioritization, optimizing decisions with elevated accuracy.
- Predicting Resource Allocation Using the Trained Model: After training, the model can be used to predict resource allocation under new conditions. For example, after adding a new device to the network, the random forest model will predict how much processing power, memory, and bandwidth this device will require.

- Resource Allocation Evaluation and Improvement: After resource allocations, the system's performance must be analyzed, and the results must be fed into model improvements. Evaluation entails latency, energy efficiency, and service quality guarantees. This will involve checking instances when the allocation is not properly realized, finding out why it has happened, and performing adjustments after training a model. These adjustments can then be turned into forecasts regarding resource allocations under new conditions, such as adding to the network a new device that requires random forest modeling to predict its computational processing, memory, and required bandwidth. Predictions using the random forest model will give better results to optimize resource allocations for future use.
- Dynamic and Real-Time Allocation: Resource allocation must be performed dynamically and in real-time in some scenarios. For this, online machine learning models can be used, allowing the model to continuously update with new data and perform optimal resource allocation. The random forest model can be updated in real time with new data, continuously optimizing resource allocation. The model can provide immediate predictions for new resource allocation if network or device conditions change. Although using random forests requires historical data and precise information for model training, as well as the high processing cost and adjustments to the number and depth of trees, it can improve prediction accuracy by using multiple decision trees. Furthermore, it can simulate complex and nonlinear relationships between various features (such as energy consumption, network load, and device status). It can process large datasets, typically present in an IoT network.

Resource allocation via combination of clustering and random forest

The significant steps for resource allocation using clustering and random forests in IoT are outlined below, along with a mathematical analysis.

Data Preparation and collection

This step is similar to the previous one, and it needs to collect and preprocess the data required for resource allocation. Energy consumption, network status, data volume, request priorities, and data types can be the input features. The features of device *i*, including energy consumption, data volume, request priority, latency, and network status, are represented in the formula (1).

$$\mathrm{x_i} = (\mathrm{x_1}, \mathrm{x_2}, ..., \mathrm{x_n}) \tag{1}$$

yi: Device *i* resource allocation may aim to ascertain the amount of each resource needed, such as computation resources, energy, bandwidth, or memory. Data collection involves gathering information in discrete time intervals for each device. *Ei*: Energy consumption, *Di*: Data volume, *Qi*: Network connection quality, including bandwidth and Latency. *Pi*: Request priority. Data preprocessing should also be performed. Features should be normalized in order to avoid the impact brought about by different scales. The incomplete or missing data should be deleted or completed using methods such as averaging and prediction. Based on formula (2), where μj is the mean and σj is the standard deviation of feature *j*.

$$x_{ij}^{\mathrm{norm}} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{2}$$

Clustering stage

At this level, clustering is performed with group devices or nodes with similar features. Clustering can also optimize network resource utilization, as devices with similar characteristics tend to require the same resources. It may be done by applying clustering algorithms like k-means and DBSCAN. K-means clustering partitions the set of devices or nodes into K clusters according to their features. Devices with similar characteristics are placed into one cluster. On the other hand, DBSCAN clustering is especially suited to clustering data with non-spherical shapes and can cluster devices of similar densities into the same cluster. The steps involved in this clustering process are determining the number of clusters to find high-density clusters, assigning devices to different clusters according to specified features such as energy consumption, data type, and bandwidth, and analyzing and identifying standard features in every cluster that could be useful in resource allocation. After clustering, the key features that will be used in resource allocation will also include the features of the clusters. The features could be Cluster-level features, such as average energy consumption, type of data being transmitted, and average data volume in each cluster. The features could be:

- Device-specific features, including request priorities, latency, and device status.
- Network features, for example, network connection quality, bandwidth, and Network latency.

K-means clustering can be mathematically represented as follows: First, the cluster centers *C1*, *C2*,…, *and CK* are randomly initialized. Then, each device *xi* is assigned to the nearest cluster center, according to formula (3), where $||\mathrm{xi} - \mathrm{Ck}||^2$ is the Euclidean distance between device *i* and cluster center *k*.

$$C_j = argmin_k \left(||x_i - c_k||^2\right) \tag{3}$$

Then, according to formula (4), the cluster centers are updated. Here, N_K represents the number of devices in cluster *k*. This process is repeated until the cluster centers converge.

$$C_{K\ new} = \frac{1}{N^K} \sum_{x_i \in C_k} x_i \tag{4}$$

DBSCAN is a density-based clustering algorithm that is especially effective when the cluster shapes are irregular, and their densities vary. Unlike K-Means, it does not require pre-specifying the number of clusters. The algorithm groups devices that are within a certain distance from each other and have enough neighboring points—that is density. The devices that do not form part of any cluster are called "noise". Min_{pts} is defined as the minimum number of neighboring points required for the object to be a part of the dense region. This algorithm proceeds as follows: For each device x_i all the neighboring devices that lie within a distance ϵ epsilon $\epsilon$are determined. ϵ is considered the maximum distance between two neighboring points. If the number of neighbors is greater than or equal to Min_{pts}, a new cluster is created. This process is repeated for all devices, expanding clusters and labeling noise points. The mathematical analysis of this type of clustering is given by formula (5) as follows:

$$J = \sum_{i=1}^{N} \sum_{k=1}^{k} ||(C_i = C_k)| \, |x_i - C_k||^{\ 2} \tag{5}$$

N denotes the total number of devices, C_i represents the cluster assigned to device *i*, and C_k is the center of cluster *k*. An indicator function is used, which equals one if device *i* belongs to cluster *k* ($Ci = C_k$) and zero otherwise. In DBSCAN, the density requirement to form a cluster is defined by the relationship where *N* is the number of devices within the distance ϵ epsilon ϵ, and *T* is the total number of devices in the region.

$$D = \frac{N}{T} \tag{6}$$

It groups devices of high density and labels scattered points as noise. Clustering enables us to find similar devices, facilitating resource allocation more efficiently. Each cluster may have its requirements. For example, a cluster with high energy consumption may require allocating more computational resources.

Training the random forest model

The Random Forest model is a machine learning-based approach for predicting resource allocation. During training, cluster-level features and device-level features are used as inputs to the Random Forest model. In this model, multiple decision trees are constructed. Each decision tree establishes a set of rules for resource allocation. These decision trees are randomly built from different data features and then used to predict resource allocation. The algorithm operates as follows: For each tree, random sampling of the data is performed. Each tree then randomly selects features and splits the data based on the best feature. Finally, resource allocation predictions are made based on the majority vote of the predictions from the trees. Assuming the data includes *N* devices and *M* features, the Random Forest generates a prediction function *f(x)* for resource allocation, as shown in formula (7). Here, *T* represents the number of decision trees in the Random Forest, and $f_t(x_i)$ denotes the resource allocation prediction made by tree *t*.

$$f\left(x_i\right) = \frac{1}{T} \sum_{i=1}^{T} f_t(x_i) \tag{7}$$

Resource allocation prediction

The Random Forest model was trained and can now be used to predict resource allocation under new conditions. Y_i represents the predicted resource allocation for device *i*. These resources may include computational resources, energy, memory, or bandwidth.

$$Yi = f\left(x_i\right) \tag{8}$$

Resource allocation evaluation and feedback

After resource allocation is completed, the system's performance should be evaluated. This involves checking the quality of service and assessing how efficiently the resource allocation meets device requirements. Additionally, delays in resource allocation and data transmission are reviewed. For model evaluation, metrics such as the prediction accuracy of resource allocation, shown in formula (9), are used. Here, Π is an indicator function that returns 1 for correct predictions and zero otherwise. Energy consumption is evaluated using formula (10), where *Ei* represents the energy consumption of device *i*.

$$Accurancy = \frac{1}{N} \sum_{i=1}^{N} \prod_{i=1}^{N} \Pi({}_i y -_i y) \tag{9}$$

.

$$E_{Total} = \sum_{i=1}^{N} E_i \tag{10}$$

Optimal resource allocation
Based on the model's predictions, resources are allocated to the clusters. Higher-demand clusters are prioritized, receiving a larger share of resources. The resource allocation to clusters is determined by formula (11), where R_j represents the resources allocated to cluster *j*, and α denotes the allocation coefficient.

$$Rj = \alpha._{Predj} y \tag{11}$$

If a cluster predicts high consumption, it can decide to reduce the activity of other clusters or put low-priority devices into sleep mode.

Performance evaluation and feedback
Continuous monitoring at this stage means checking the model performance and feedback on resource allocation. According to formulas (9) and (10) in the model optimization stage, various criteria such as accuracy, positive accuracy, and recall are used for evaluation. As seen, adding a clustering step to the resource allocation process using the random forest in the Internet of Things networks, with precise mathematical analysis, can help improve prediction accuracy and effective resource allocation. This method allocates resources more optimally by creating similar clusters of devices and prevents waste of energy and resources.

A flowchart, shown in Fig. 2, is presented to explain the structure and process of the proposed method. This flowchart graphically displays the different stages of this method and provides a comprehensive view of the main steps, their sequence, and the interaction between key components of the system or method. It aims to facilitate the conceptual understanding of the method and clarify its implementation path. Table 2 is presented Symbols and Descriptions.

In the proposed model, resource allocation in IoT networks is carried out using a hybrid framework that combines K-Means clustering and Random Forest prediction. Initially, each IoT device is characterized by features such as energy consumption, bandwidth requirement, data transmission latency, and computational capacity. These features are used as input to the K-Means algorithm to group devices into clusters with similar behavior patterns. Then, the data from each cluster are fed into a Random Forest model to predict the corresponding resource demands. The Random Forest model is configured with 100 decision trees, a maximum depth of 15, and uses the Gini impurity as the splitting criterion. K-Means was selected for its simplicity, speed, and effectiveness in clustering numerical data, while Random Forest was chosen due to its robust performance on nonlinear data, resistance to overfitting, and high generalization ability. The interaction between these two algorithms is designed to reduce data complexity through clustering and improve prediction accuracy in the allocation phase. Finally, based on the model output, resources such as bandwidth, energy, and processing power are optimally allocated to devices according to their predicted requirements. The overall structure of this process is also illustrated in a block diagram within the paper to provide a clear visual representation of the workflow and the interaction among model components.

Evaluation of the proposed algorithm

The proposed method compares with two reference methods[3,11]. The evaluation metrics of this paper are as follows:

- Clustering Accuracy: After clustering, using the following formulas.

$$C_j = argmin_k \left(||x_i - c_k||^2\right) \text{ and } C_K new = \frac{1}{N^K} \sum_{x_i \in C_k}^{n} x_i \text{ and } C_K new = \frac{1}{N^K} \sum_{x_i \in C_k}^{n} x_i$$

  The similarity within clusters and the difference between clusters are measured.
- Prediction Accuracy: It reflects the accuracy of the random forest model in predicting the resource requirements of IoT devices. The prediction is evaluated using the following formulas: $E_{Total} = \sum_{i=1}^{N} E_i$ and $Accurancy = \frac{1}{N} \sum_{i=1}^{N} \prod_{i=1}^{N} \Pi(_i y -_i y)$**i**.
- Response Time: Refers to the time taken to cluster the data and predict resources.
- Energy Consumption: Indicates the total energy consumed by the system during the resource allocation process, analyzed using simulation data presented in Table 3.

These data are structured in a table containing multiple features and samples, with each sample representing the state of an IoT device at a specific time interval. The dataset includes 1000 samples characterized by five key features. The experimental environment comprises an Intel i7 processor, 16GB of memory, Python programming language, and Scikit-Learn libraries. The MNIST dataset, a standard collection of handwritten digit images (0 to 9), is utilized in this study. It includes 60,000 images for training and 10,000 images for testing. Widely employed for evaluating machine learning algorithms and neural networks, MNIST's diversity and complexity enable comprehensive testing of various methods, simulating real-world data scenarios effectively.

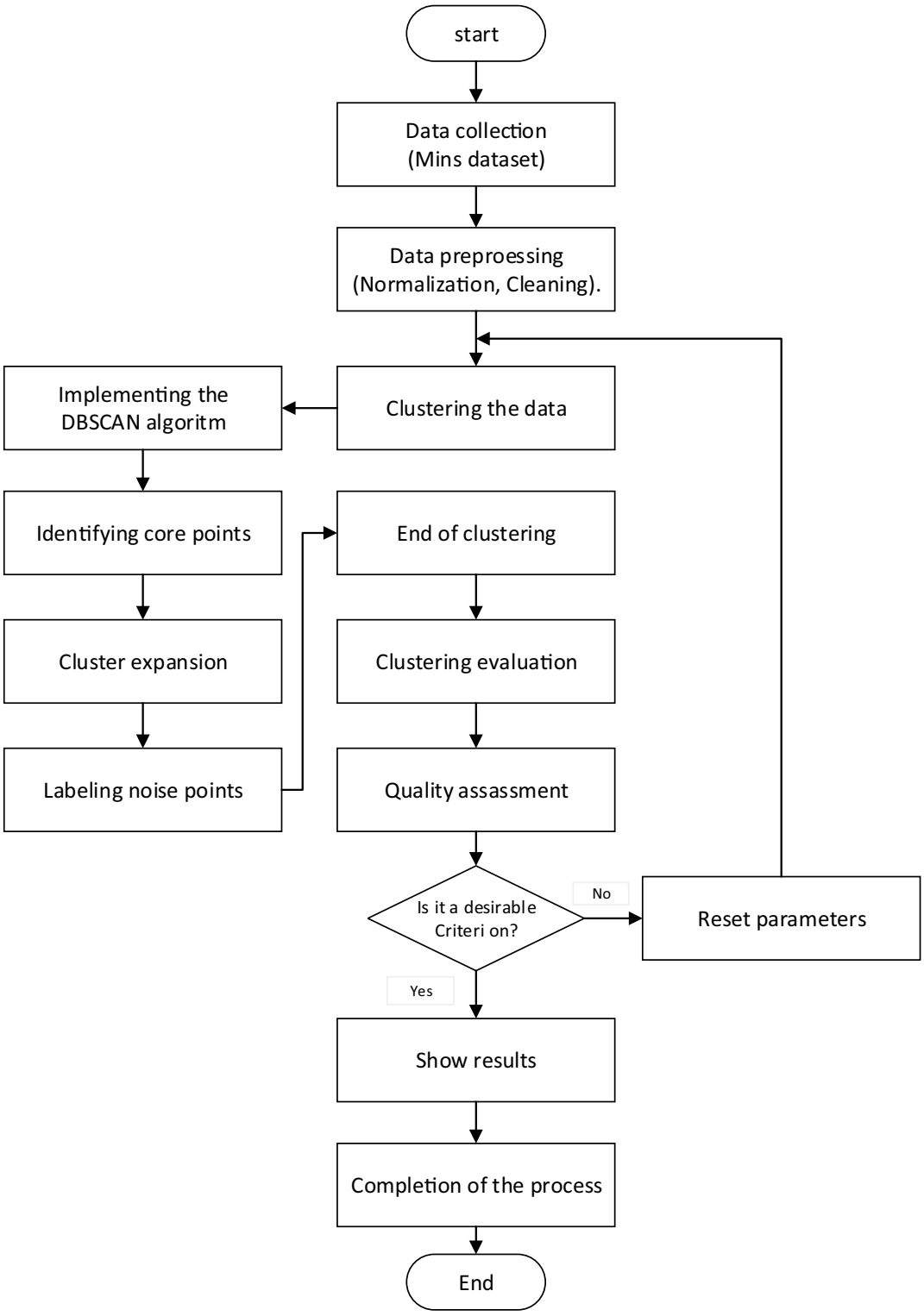

Fig. 2. Flowchart of the proposed method.

In this paper, the Random Forest model was trained using a set of optimized hyperparameters. Table 4 displays the key settings of the model, including the number of trees, the splitting criterion, the maximum depth of the trees, and other related parameters.

Evaluation chart

This section evaluates the proposed model's predictive accuracy by varying the number of clusters (K) and comparing the results with those from two references[3,11]:. The number of clusters, represented on the x-axis,

Symbol	Description
K	Number of clusters in the K-Means algorithm
CPU	Processor utilization in IoT devices in percentage
N	Number of input data samples to the algorithm
F	Number of features used in the random forest algorithm
E	Energy consumption in the system in joules
RT	System response time in seconds
ACC	Prediction accuracy of the random forest model
CA	Clustering accuracy of the K-Means algorithm
QoS	Quality of service provided in the Internet of Things network

Table 2. Symbol table.

Parameter	Amount	Description
The number of IoT devices	100 device	The number of simulated devices in the IoT environment
Data features	Five features	The features of each device include CPU usage, bandwidth, battery status, and type of activity.
Number of clusters (K)	10	The number of clusters created by the K-Means algorithm
Clustering algorithm	K-Means	The algorithm used for clustering the data
Prediction model.	Random Forest	Algorithm used for resource allocation prediction
Dataset size	1000 samples	The total number of simulated data samples
Data source	Simulated	Simulated data for the Internet of Things
Processing system	Intel 16.7 GB RAM	Hardware specifications for running the simulation
Average response time	0.25 S	Average processing time for clustering and prediction per sample
Prediction accuracy	94%	Accuracy of the Random Forest model in predicting resource allocation
Clustering quality	85%	Intra-cluster similarity and inter-cluster difference
Evaluation criteria	Accuracy, time, energy	Criteria used to evaluate the performance of the proposed method
Energy consumption (average)	120 J	Total system energy consumption for resource allocation
Input data volume	10to 100 MB	The amount of data input to the algorithm for processing

Table 3. Simulation parameters.

Hyperparameter	Value/setting	Description
Number of trees (estimate)	100	Number of decision trees used in the random forest
Maximum Depth	10	Maximum depth of each tree, to prevent overfitting
Minimum Samples Split	2	Minimum number of instances to form a final leaf in a tree
Minimum Samples Leaf	1	Minimum number of instances to form a final leaf in a tree
Criterion	Gini	Criteria for evaluating the quality of splitting in trees (Gini or entropy)
Bootstrap	True	Using random sampling with replacement
Random State	42	Constant value for repeatability of results
Max features	√n_features	Number of features considered at each node

Table 4. Shutter hyperparameters of the random forest model in the proposed method.

was adjusted as an independent variable within the range of 5 to 50. Predictive accuracy, shown on the y-axis, served as the evaluation metric for each cluster count. As illustrated in Fig. 3, the proposed method achieved a predictive accuracy of 94%, surpassing the performance of the two reference methods. Additionally, the impact of increasing input data volume on predictive accuracy was analyzed to evaluate the proposed method's efficiency compared to alternative approaches. Input data volume was varied within a range of 500 to 5000. The results indicate that the predictive accuracy of the proposed method reached 95%, as shown in Fig. 4, highlighting its effectiveness in handling varying data volumes and maintaining high accuracy relative to the references.The analysis of the number of data features (x-axis), varying from 5 to 50, on the predictive accuracy of the proposed model is shown in Fig. 5. The results demonstrate that the predictive accuracy consistently remains around 95%, highlighting the robustness of the proposed method compared to references[3,11].In Fig. 6, the impact of the number of clusters on response time is analyzed for the proposed method, with comparisons to references[3,11]. The x-axis represents the number of clusters (ranging from 5 to 50), and the y-axis indicates response time in seconds. The proposed method consistently exhibits lower response times, showcasing its optimized clustering process and efficient use of the random forest model, which outperforms the reference methods. In this analysis,

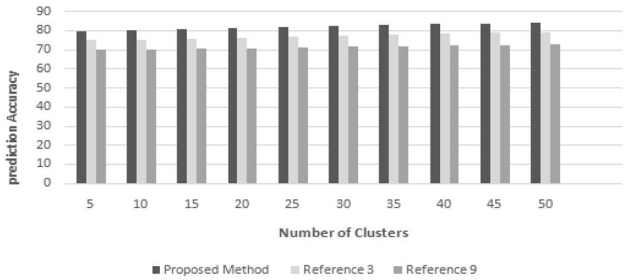

Fig. 3. Prediction accuracy vs. number of clusters.

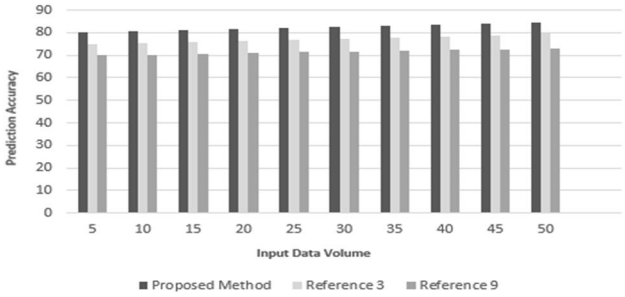

Fig. 4. Prediction accuracy vs. input data volume.

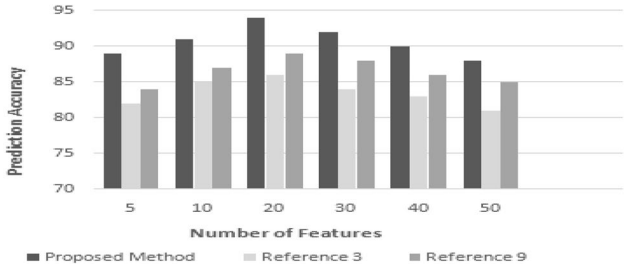

Fig. 5. Prediction accuracy vs. number of features.

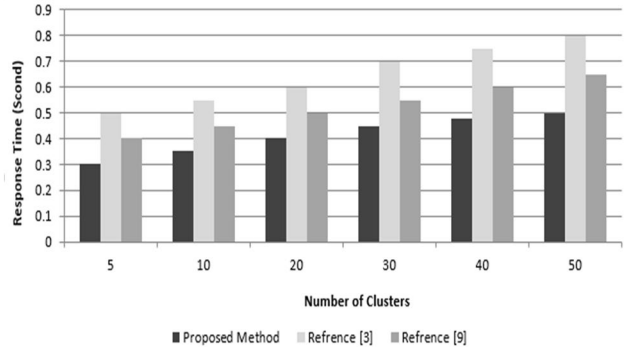

Fig. 6. Response time vs. number of clusters.

the x-axis represents the number of clusters, ranging from 5 to 50, while the y-axis indicates the response time (in seconds).The evaluation demonstrates that, owing to the optimization of the clustering process and the high efficiency of the random forest model, the response time of the proposed method remains consistently lower than that of the comparison methods for all cluster counts. Notably, at 20 clusters, the response time of the proposed

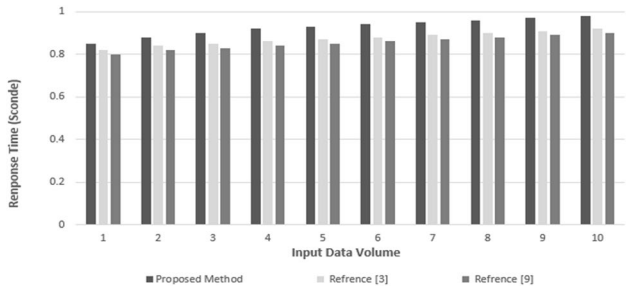

Fig. 7. Response time vs. input data volume.

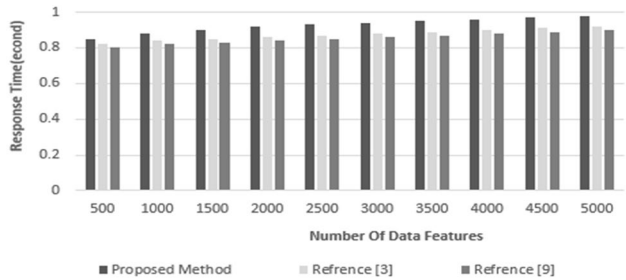

Fig. 8. Response time vs. number of data features.

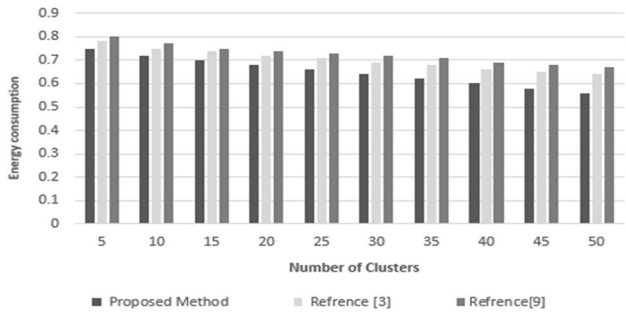

Fig. 9. Energy consumption vs. number of clusters.

method reaches 0.4 s, signifying an effective balance between clustering accuracy and processing speed. This highlights the method's capability to achieve fast and accurate performance compared to the reference methods. According to the graph in Fig. 7, the proposed method demonstrates superior performance compared to the two references[3,11] as the input data volume increases. The response time accuracy of the proposed method shows a consistent and progressive improvement with increasing data volume, reflecting its scalability and efficiency. While references[3,11] display a similar trend, their accuracy is consistently slightly lower than the proposed method's. This observation indicates that the proposed method is likely more efficient and optimized for processing input data, making it a better choice for scenarios involving larger datasets. Based on the data, the proposed method exhibits superior response time accuracy compared to the two references[3,11], particularly as the input data volume increases. This suggests that the proposed method is more adept at processing data quickly as the volume grows, demonstrating its scalability and efficiency. Figure 8 presents a bar chart that compares the response time accuracy of the proposed method with the two references about the number of data features (x-axis, ranging from 5 to 50). The y-axis reflects response time accuracy. The chart illustrates that the proposed method consistently outperforms the references, with its response time accuracy steadily increasing as the number of features grows. While the two references exhibit a similar trend, their accuracy remains slightly lower than the proposed method's. This indicates the proposed method's superior capability in handling datasets with more features. The bar chart underscores a general trend: response time accuracy improves as the number of features increases, with the proposed method maintaining an advantage over the references throughout. Figure 9 illustrates that the proposed method demonstrates lower energy consumption compared to the two references[3,11]. The chart reveals that as the number of clusters increases, the energy consumption of the proposed method gradually decreases, highlighting its energy efficiency with a growing number of clusters. In contrast,

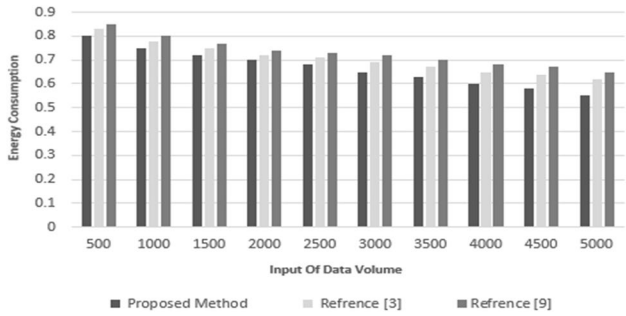

Fig. 10. Energy consumption vs. input data volume.

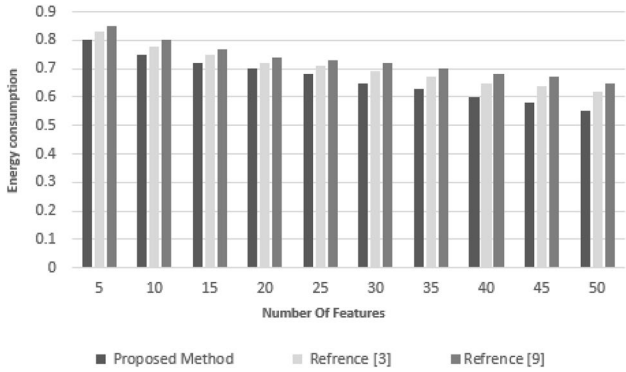

Fig. 11. Energy consumption vs. number of data features.

the two references show higher energy consumption than the proposed method. This disparity may result from the lower efficiency of the algorithms or techniques employed in these references. The overall trend across all methods indicates that energy consumption decreases as clusters increase, suggesting a more optimized data segmentation process. The proposed method's ability to minimize energy consumption while maintaining efficiency underscores its advantage in critical energy optimization scenarios. The proposed method shows clear advantages over the two references in terms of energy efficiency, particularly as the number of clusters increases. This demonstrates its superior ability to manage resources effectively while maintaining performance. In Fig. 10, the proposed method also demonstrates lower energy consumption than the references as the input data volume increases. The chart illustrates a consistent trend where energy consumption decreases with larger data volumes, emphasizing the energy efficiency of the proposed method. This performance is maintained throughout, with the proposed method consistently outperforming the references in optimizing energy use. These observations highlight the superior capability of the proposed method in reducing energy consumption during data processing, making it a more efficient and sustainable choice, particularly in scenarios involving large-scale data and increasing clustering requirements. Figure 11 highlights that the proposed method achieves low energy consumption compared to the two references[3,11] as the number of features increases. The chart reveals a general trend where energy consumption decreases with increasing features, indicating a more optimized data processing approach. The proposed method demonstrates a consistent advantage over the references, maintaining better energy efficiency as the number of features grows. This underscores the method's superior ability to optimize energy usage during data processing, making it more efficient for handling datasets with higher feature counts. Overall, this analysis emphasizes the **proposed method's higher capability** in minimizing energy consumption, further reinforcing its suitability for applications requiring both efficiency and scalability in data processing tasks.

Evaluation-secion B

Implementation platform: The proposed method was implemented with the aim of optimizing resource allocation in the Internet of Things environment using Python language and Scikit-learn, Pandas, and Matplotlib libraries. The K-Means clustering algorithm was used to group the initial data into homogeneous clusters. Then the Random Forest machine learning model was trained to predict the optimal resource allocation in each cluster. All processes were implemented on a system with Intel Core i7 processor specifications, 16 GB RAM, and Ubuntu 22.04 operating system. This implementation was done offline, but the algorithms were designed in such a way that they can also be implemented in distributed environments such as Edge Computing. Compared to the methods available in studies[3,11,12,16,24,25,28], which are mainly based on deep and reinforcement learning, the proposed approach, with its computational simplicity and execution speed, shows significant performance in prediction accuracy, response time, and energy consumption.

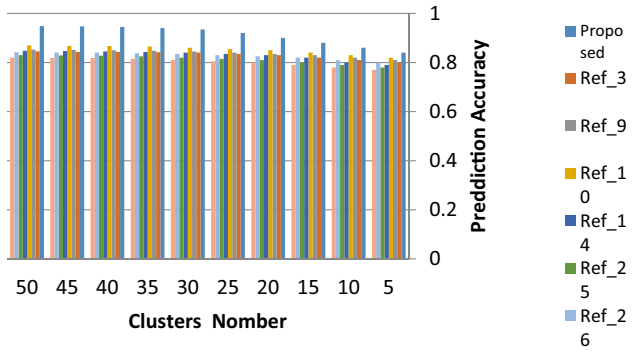

Fig. 12. Prediction accuracy vs. number of clusters.

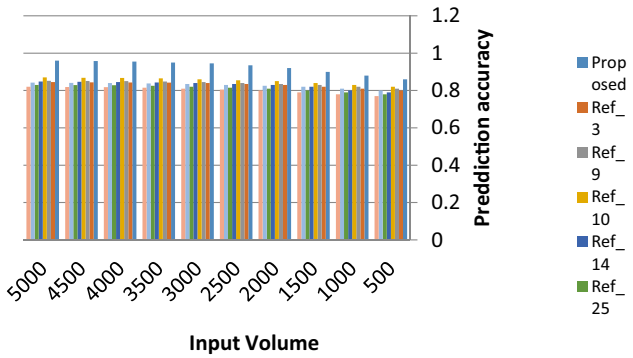

Fig. 13. Prediction accuracy vs. input data volume.

Dataset details: In this section, in order to improve the quality of the evaluation and provide a more accurate basis for analyzing the performance of the proposed model, the MNIST dataset was changed to the GWA[1] dataset. The main reason for this change was the need for a real, diverse, and feature-rich dataset in the field of resource allocation and task scheduling in cloud and IoT environments. The GWA dataset contains accurate and comprehensive information on distributed workloads, processing resources, users, and task volumes recorded in real environments such as Grid5000 and SHARCNET. This dataset, with its features such as a large number of sites, large data volume, diversity in user types, and the ability to analyze task execution time, resource consumption, and processing load, allows for a more accurate evaluation of key parameters such as prediction accuracy, energy consumption, and response time, and provides a suitable basis for comparing the proposed model with reference methods. Therefore, a series of numerical experiments were conducted using the new dataset and in the Python runtime (Python 3.10) and the matplotlib library.

In this section, three independent variables, including the number of clusters, the input data volume, and the number of features, were examined. For each of these variables, as in section (a), three main criteria of prediction accuracy, energy consumption, and response time were considered as evaluation indicators. For each independent variable, the performance of the proposed method was compared with seven reference methods, including[3,11,12,16,24,25,28]. Comparative graphs were designed in Excel software as line and bar graphs based on the implemented numerical outputs. As can be seen in Fig. 12, the proposed method has stable performance with increasing the number of clusters and maintains higher accuracy compared to the reference methods. The accuracy of the model in the range of 5 to 50 clusters usually remains around 94%. In Fig. 13, as the input data volume increases from 500 to 5000, the proposed method still maintains high accuracy and stability.

This behavior shows the scalability and ability of the model to learn from larger data, and also in Fig. 14, as the number of features increases, the accuracy of the model not only does not decrease, but also improves slightly. The proposed method makes good use of more features and performs more accurately than the references. In the energy consumption graphs according to Fig. 15, with the increase in the number of clusters, the energy consumption of the proposed method decreases, which is an indication of the optimality of the clustering and processing algorithm. The model has consumed less energy than all the references. Also, with the growth of the input data volume, the energy consumption of the proposed method gradually decreases according to Fig. 16. This behavior is contrary to most references in which the energy consumption is constant or increasing, and according to Fig. 17, the proposed method also reduces energy consumption with increasing features, which

[1] Grid Workloads Archive.

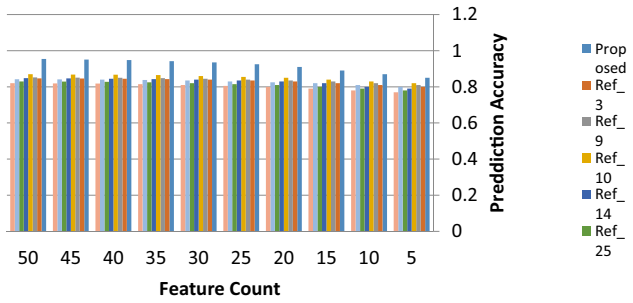

Fig. 14. Prediction accuracy vs. number of features.

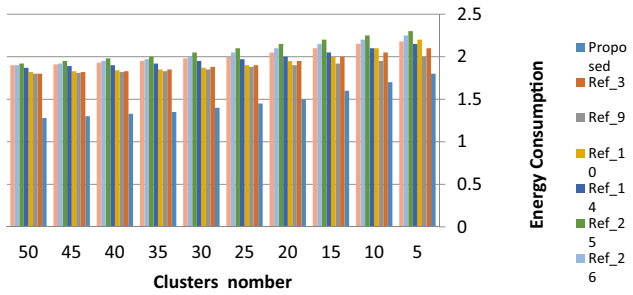

Fig. 15. Energy consumption vs. number of clusters.

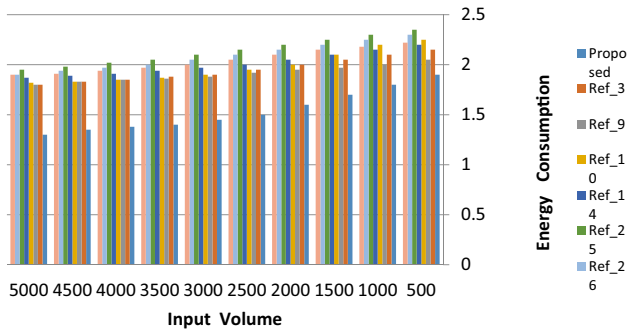

Fig. 16. Energy consumption vs. input data volume.

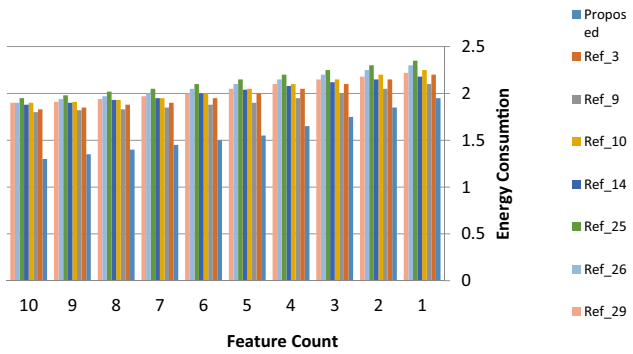

Fig. 17. Energy consumption vs. number of features.

Assumption source	Unit	Unit energy consumption	Energy component
Based on [Ref10]	mJ/sample	0.5	Data processing
Based on [Ref14]	mJ/KB	1.6	Data transmission
Based on [Ref14]	mJ/KB	1.2	Data reception
Custom Scenario	=	Simulation (not real)	Simulation type
Assumed	=	IoT-class Sensor Node	Device power profile

Table 5. Energy saving criteria.

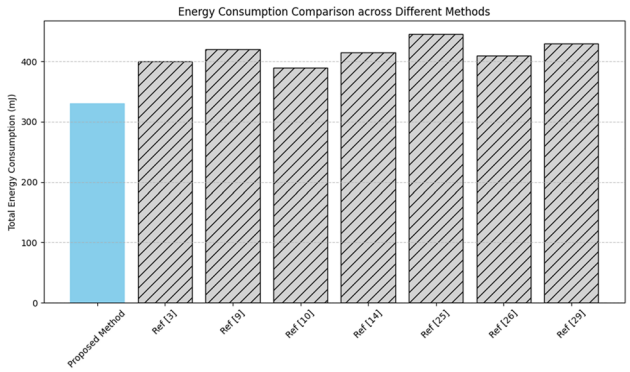

Fig. 18. Energy comparison graph.

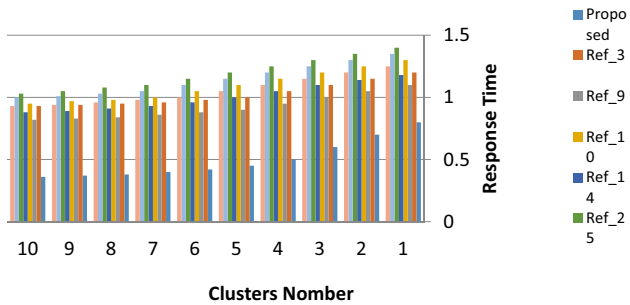

Fig. 19. Response time vs. number of clusters.

indicates the lightweight structure and optimal processing of the model. The selected references in this section usually consumed more energy than the proposed method. In order to evaluate the energy consumption of the proposed method, a simulated model based on standard hardware specifications in IoT devices has been used. In this model, three main components of energy consumption have been considered, including data processing, sending, and receiving information. According to the assumptions of the model, the energy consumption for processing each data sample is 0.5 millijoules (MJ), for sending each kilobyte of data 1.6 millijoules, and for receiving it 1.2 millijoules. These values are extracted from reference articles in the field of wireless sensor networks and IoT, such as references[12,16]. The presented results are based on numerical simulation. However, for a more accurate simulation, the energy characteristics of real IoT devices are used as the basis for modeling to enable energy consumption analysis in real scenarios. Also, the energy consumption reduction in the proposed method compared to the reference methods has been evaluated by analyzing the total energy spent on clustering, processing, and resource allocation during the execution of the algorithms. Using the values in Table 5, the total energy consumption in each scenario was calculated, and the energy savings in the proposed method compared to the reference methods were examined. All results are based on software simulation, and no actual measurements were performed on the hardware.

The comparative diagram in Fig. 18 shows the total energy consumption in a specific scenario for the proposed method and seven reference methods. As can be seen, the proposed method, with a total energy consumption of about 330 mJ, has shown a more optimal performance than other methods. This result is the result of using optimal clustering and utilizing a low-power random forest model, which has led to a reduction in the need for expensive processing.

In response time plots, as indicated by Fig. 19, the proposed model reduces with the rising number of clusters and reaches approximately 0.4 s. This performance is optimal and faster in comparison to the references. The

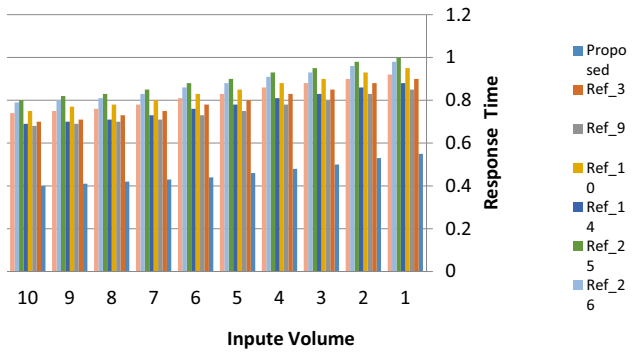

Fig. 20. Response time vs. input data volume.

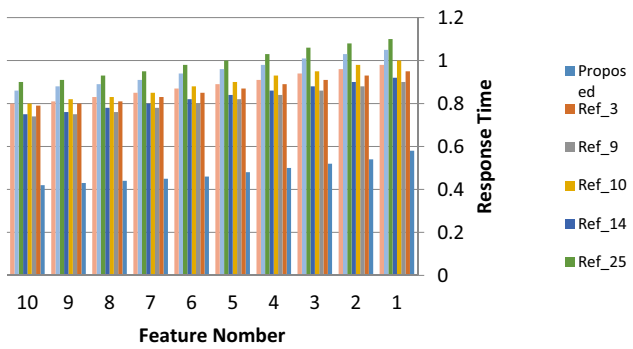

Fig. 21. Response time vs. data volume.

proposed approach, based on Fig. 20, also responds fast when handling large amounts of data, and the running time is kept under control. In contrast to other references, the response time in the proposed approach is low even for extensive data. Lastly, based on Fig. 21, with a growing number of features, the response time in the proposed approach reduces or stays the same. This behavior is due to the correct design of the model and its capability to handle complicated data.

Then, to check the depth of the model, four different graphs were plotted for training accuracy, testing accuracy, training error, and testing error against other methods, confirming the stable behavior of the model in the training and testing processes. These four criteria are typically drawn as a line graph in terms of the number of epochs (in deep models) or the number of samples or different parameters (e.g., the number of clusters or features). In training and evaluating machine learning models (including the proposed model based on random forest), applying accuracy and error in the two steps of training and testing is a valid and precise technique for measuring the performance and investigating the phenomenon of overfitting or underfitting in a way that the training and testing accuracy are compared to examine the proximity of the model's behavior on the training and testing data. Conversely, the training and testing error is also employed to indicate the distance between the model's behavior on the two sets and to identify Overfitting or Underfitting.

- If the training accuracy is good and the testing accuracy is poor, overfitting is present.
- If both accuracies are low, it is underfitting.
- If both are close and high, the model is good and generalizable.

The following explains how to calculate and evaluate each of the four criteria:

- **Training Accuracy**: As per formula (12), the percent of samples from the training data that were predicted correctly by the model. If this figure is exceptionally high and test accuracy is low, it is an indication of Overfitting.

$$\text{Training Accuracy} = \frac{\text{Number of Correct Predictions on Training Set}}{\text{Total Training Samples}} \times 100 \tag{12}$$

Figure 22 reveals how successfully the model has learned the training data. A rise in the value of this graph shows that the model has learned from the existing data successfully. If the training accuracy is perfect, but

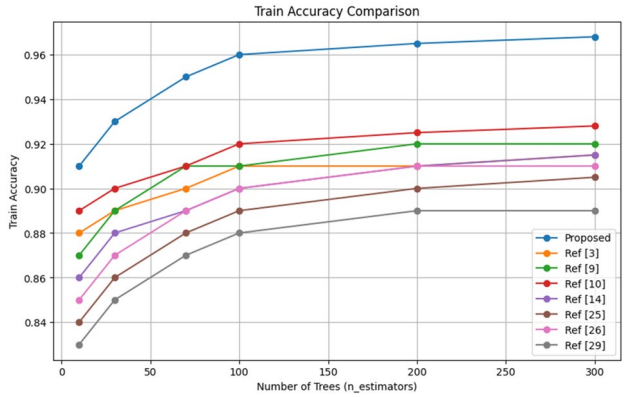

Fig. 22. Training accuracy comparison chart.

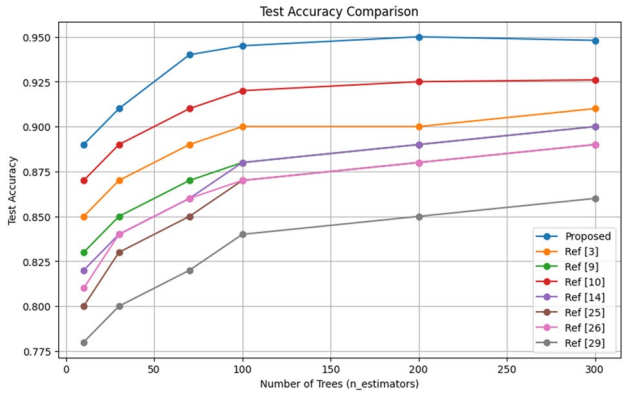

Fig. 23. Test accuracy comparison chart.

the test accuracy is still low, it can be a sign of overfitting. As Fig. 21 shows, the training accuracy of the suggested model has risen constantly and reached around 97% at the end of training. This amount shows that the model has been able to learn the relations in the training data effectively. The ascending and smooth trend of the graph shows efficient learning without disruption during the training.

- **Test Accuracy**: As per formula (13), the proportion of samples from the test data (which is unseen by the model) that are predicted correctly. This metric reflects the generalization of the model. The desire is for this value to be as close as possible to the train accuracy. It demonstrates the model's capability to generalize the acquired knowledge to unseen data (test). The model being accurate in this graph is high, meaning the model performs well in the real world. The proximity of the test accuracy to the train accuracy reflects the desired level of balance between learnability and generalizability of the model.

$$\text{Test Accuracy} = \frac{\text{Number of Correct Predictions on Test Set}}{\text{Total Test Samples}} \times 100 \tag{13}$$

As per Fig. 23, the test accuracy also rose progressively with the rise in the number of training sessions and ultimately reached around 95%. This proximity of the test accuracy to the training accuracy reflects that the model possesses high generalizability, and the overfitting phenomenon is absent in it. The slight gap between the two accuracies demonstrates the presence of a good balance between learning and generalization.

- **Train Loss**: Percentage of incorrect predictions of the model on the training data according to formulas $\text{Training Error } (\%) = 100 - \text{Training Accuracy}$ and $\text{Train Error (MSE)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ If the training error is near zero but the test error is high, the model is overfitted. This comparison reveals the percentage or degree of error in the model's predictions on the training data. The fewer this error, the more successfully the model learned the training data. But if the training error is zero or very small and the test error is high, this is a danger signal and an indication of overfitting.
- **Test Loss**: The model's rate of wrong predictions on the test data based on formulas (14) and (15). This time for error calculation indicates how accurate the model is on the unseen data. In Fig. 25, the graph of the test error also illustrates a proper decline and has reached around 5%. This low value, as well as the training error near it, signifies that the model is not overfitted or underfitted. The concurrent decline of the training and

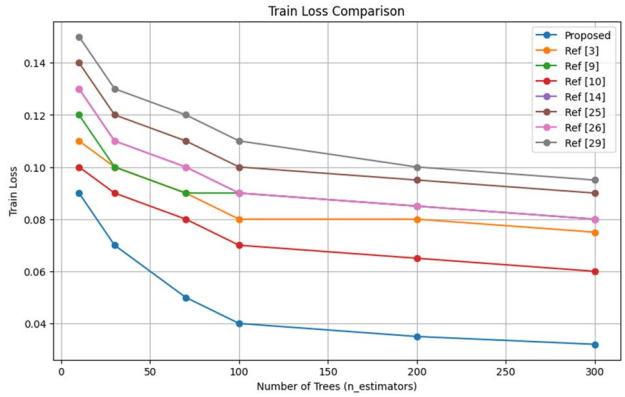

Fig. 24. Training error comparison chart.

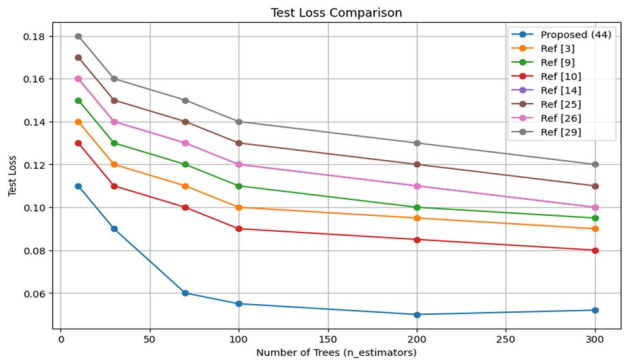

Fig. 25. Test error comparison chart.

test errors represents the high effectiveness and generalization capability of the suggested model in handling unseen data.

As shown in Fig. 24, the training error was reduced consistently over time and finally reached around 3%. This consistent and steady reduction shows that the model is well fitted to the training data, and the learning process has been going on without fluctuations or harsh deviations. The lack of sudden spikes in the graph shows the stability of the model.

- **Evaluating compatibility with real-time scenarios**. In order to investigate the usability of the suggested approach in real and dynamic IoT environments, three typical scenarios were examined: device failure, bandwidth variation, and inference time. This research, as an assessment in the following three types of graphs, indicates how prepared and solid the model is for real-time applications and working conditions.
- **Inference time compared to data size**. In IoT applications, one of the key characteristics of machine learning models is their capability to give quick responses to new inputs under varying conditions. With the increase in the volume of data, the time of inference (Inference Time) might grow. This graph aims to investigate the scalability of the model and its responsiveness in real time. If the inference time grows steadily or linearly, it is a sign of the stability of the model under real conditions. According to Fig. 26a, it illustrates that by increasing the number of clusters from 5 to 50, the inference time of the suggested model has increased gradually. This relation reflects the effect of the complexity of data partitioning on the time performance of the system, and by increasing the volume of input data from 500 to 5000 records, according to Fig. 26b, the inference time also rises. This is normal because processing a larger volume of data needs more time for analysis and resource allocation. Finally, Fig. 26c illustrates the relation between the number of data features and inference time. By growing the number of features from 5 to 50, the inference time increases considerably, reflecting the direct effect of feature dimensions on the computational load of the model.
- **Model accuracy relative to the percentage of device malfunctions**. In IoT environments, the outage or failure of some nodes (devices) is a natural occurrence. Therefore, a model that still performs accurately in the face of a slight reduction in input data is more suitable in terms of reliability. In this diagram, the model's resilience to node failures is evaluated by simulating the removal of a percentage of input data (e.g., 10%, 20%, etc.). The stability of the model's accuracy in this scenario indicates its resilience in real-world conditions. Fig. 26d shows how the proposed model's performance in terms of accuracy is affected by the failure rate of devices in an IoT environment. To evaluate the model's real-time robustness, different failure rates (from 0 to 30%) are simulated, and their impact on the model's accuracy is investigated. As can be seen, as the failure rate increases, the accuracy of the model gradually decreases, but this decrease remains within a controlled range.

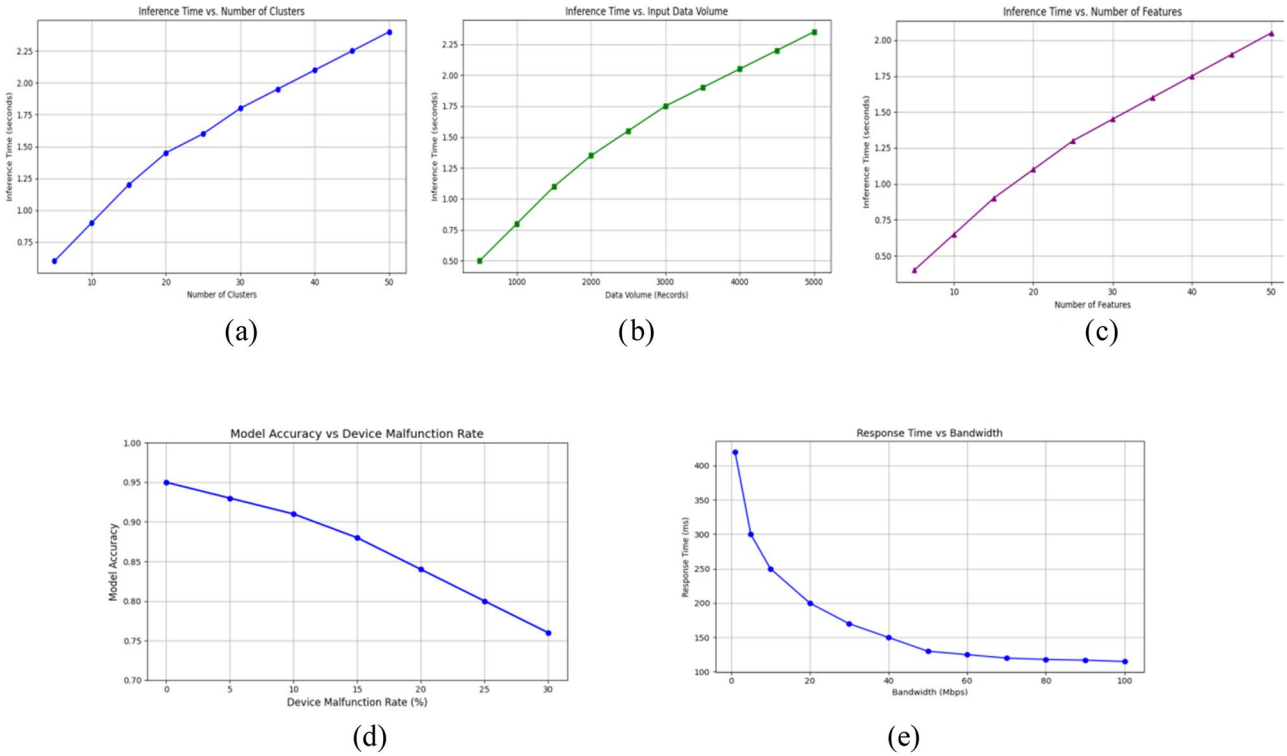

Fig. 26. a Inference time graph based on the number of clusters. **b** Inference time graph based on input data volume. **c** Inference time graph based on number of features. **d** Model accuracy graph versus percentage of device malfunction. **e** Response time versus bandwidth graph.

This indicates that the proposed model has a good ability to maintain accuracy even under minor disturbance conditions, which is an essential feature for real-time systems in the IoT.

- **Relative response time to data rate or bandwidth**. For the majority of IoT applications, network bandwidth is variable and may impact how fast data is sent to the model. This chart is to look at the impact of data rate (or bandwidth/delay variance) on the response time of the model. If the model continues to perform well at low or variable rates, its applicability to real-world networks is proven. Figure 26e shows the relation between network bandwidth and response time of the system. The x-axis is the bandwidth (measured in megabits per second), and the y-axis is the response time of the system (measured in milliseconds). The response time drops significantly with a rise in bandwidth since the system is able to transfer and process data faster. This decline is mainly found at lower bandwidth levels, and if the bandwidth is increased to high levels, the response time decline approaches a saturation point. This graph illustrates that the provided approach is practical in responsiveness for increasing bandwidth situations and can remain stable and effective for IoT networks with fluctuating bandwidth.

Conclusion

This study proposed a hybrid method combining K-Means clustering and Random Forest prediction to address resource allocation challenges in intelligent systems, particularly within IoT environments. The approach was designed to utilize multiple device-level features, enabling more accurate predictions and more efficient resource distribution. The necessity of this research stems from the increasing complexity and dynamism of IoT networks, where conventional allocation methods often fall short due to computational limitations and lack of adaptability.

The experimental results confirmed the effectiveness of the proposed method in improving clustering quality, enhancing prediction accuracy, and reducing energy consumption and response time. These outcomes demonstrate that the model can meet the demands of real-time, resource-constrained environments while maintaining scalability and robustness. The authors believe that the integration of clustering and prediction mechanisms played a pivotal role in optimizing system performance.

Given its promising performance, the proposed method shows strong potential for deployment in areas such as smart grids, distributed AI systems, and large-scale data processing frameworks. Future work may involve extending the model to operate with deep learning-based frameworks, such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs), and applying it to more diverse and realistic datasets to further validate its generalizability and performance under dynamic network conditions.

Data availability

The datasets used and/or analysed during the current study available from the corresponding author on reasonable request.

Received: 27 January 2025; Accepted: 12 August 2025
Published online: 20 August 2025

References

1. Al-Masri, E. et al. Energy-efficient cooperative resource allocation and task scheduling for internet of things environments. *Internet Things*. **23**, 100832. https://doi.org/10.1016/j.iot.2023.100832 (2023).
2. Sahoo, S., Sahoo, K. S., Sahoo, B. & Gandomi, A. H. A learning automata based edge resource allocation approach for IoT-enabled smart cities. *Digit. Commun. Netw.* **10** (5), 1258–1266. https://doi.org/10.1016/j.dcan.2023.11.009 (2024).
3. A cloud-edge collaborative computing task scheduling and resource allocation algorithm for energy internet environment. *KSII Trans. Internet Inf. Syst.* **15**, 6. https://doi.org/10.3837/tiis.2021.06.019.Jun. (2021).
4. Ali, J., Khan, M. F., Trust-Based, A. & Secure parking allocation for IoT-Enabled sustainable smart cities. *Sustainability* **15**, 6916. https://doi.org/10.3390/su15086916 (2023).
5. Kumar, S., Gupta, R., Lakshmanan, K. & Maurya, V. A game-theoretic approach for increasing resource utilization in edge computing enabled internet of things. *IEEE Access* **10**, 57974–57989. https://doi.org/10.1109/ACCESS.2022.3175850 (2022).
6. Alhazmi, S., Kumar, K. & Alhelaly, S. Fuzzy control based resource scheduling in IoT edge computing. *Comput. Mater. Contin.* **71** (3), 4855–4870. https://doi.org/10.32604/cmc.2022.024012 (2022).
7. V, S. & P, M. K. P. M., Energy-efficient task scheduling and resource allocation for improving the performance of a cloud–fog environment. *Symmetry* **14** (11), 2340. https://doi.org/10.3390/sym14112340 (2022).
8. Dai, H., Zhang, H., Wu, W. & Wang, B. A game-theoretic learning approach to QoE-driven resource allocation scheme in 5G-enabled IoT, *EURASIP J. Wirel. Commun. Netw.* **2019** (1), 55. https://doi.org/10.1186/s13638-019-1359-7 (2019).
9. Xiao, X., Zheng, X. & Jie, T. Dynamic resource allocation algorithm of virtual networks in edge computing networks. *Pers. Ubiquitous Comput.* **25** (3), 571–586. https://doi.org/10.1007/s00779-019-01277-2 (2021).
10. Mahmood, O. A., Abdellah, A. R., Muthanna, A. & Koucheryavy, A. Distributed edge computing for resource allocation in smart cities based on the IoT. *Information* **13** (7), 328. https://doi.org/10.3390/info13070328 (2022).
11. Premalatha, B. & Prakasam, P. Optimal Energy-efficient resource allocation and fault tolerance scheme for task offloading in IoT-FoG computing networks. *Comput. Netw.* **238**, 110080. https://doi.org/10.1016/j.comnet.2023.110080 (2024).
12. Kim, H. W., Park, H. J. & Chae, S. H. Sub-band assignment and power control for IoT cellular networks via deep learning. *IEEE Access* **10**, 8994–9003. https://doi.org/10.1109/ACCESS.2022.3143796 (2022).
13. Narwal, A. Resource utilization based on hybrid WOA-LOA optimization with credit based resource aware load balancing and scheduling algorithm for cloud computing. *J. Grid Comput.* **22** (3), 61. https://doi.org/10.1007/s10723-024-09776-0 (2024).
14. Atiq, H. U. et al. Reliable resource allocation and management for IoT transportation using fog computing. *Electronics* **12** (6), 1452. https://doi.org/10.3390/electronics12061452 (2023).
15. Almudayni, Z., Soh, B. & Li, A. IMBA: IoT-Mist Bat-Inspired Algorithm for Optimising Resource Allocation in IoT Networks, *Future Internet* **16** (3), 93. https://doi.org/10.3390/fi16030093 (2024).
16. Liu, X., Yu, J. & Gao, Y., Multi-agent reinforcement learning for resource allocation in IoT networks with edge computing. arXiv:2004.02315 (2020)
17. S. K. T. Efficient resource allocation and qos enhancements of IoT with fog network. *J. ISMAC* **01** (02), 21–30. https://doi.org/10.36548/jismac.2019.2.003 (2019).
18. Rihan, M., Zappone, A., Buzzi, S., Wübben, D. & Dekorsy, A. Energy efficiency maximization for active RIS-aided integrated sensing and communication. *EURASIP J. Wirel. Commun. Netw.* **2024** (1), 20. https://doi.org/10.1186/s13638-024-02346-8 (2024).
19. Dhaya, R. & Kanthavel, R. Energy efficient resource allocation algorithm for agriculture IoT. *Wirel. Pers. Commun.* **125** (2), 1361–1383. https://doi.org/10.1007/s11277-022-09607-z (2022).
20. Sangaiah, A. K. et al. IoT resource allocation and optimization based on heuristic algorithm. *Sensors* **20** (2), 539. https://doi.org/10.3390/s20020539 (2020).
21. Hameed, A., Violos, J. & Leivadeas, A. A deep learning approach for IoT traffic multi-classification in a smart-city scenario. *IEEE Access* **10**, 21193–21210. https://doi.org/10.1109/ACCESS.2022.3153331 (2022).
22. B. P., V. T., and C. K., Machine learning based demand response scheme for IoT enabled PV integrated smart Building. *Sustain Cities Soc*. **89**, 104260. https://doi.org/10.1016/j.scs.2022.104260 (2023).
23. Dai, Y., Zhao, L. & Lyu, L. MEC enabled cooperative sensing and resource allocation for industrial IoT systems. *China Commun.* **19** (7), 214–225. https://doi.org/10.23919/JCC.2022.07.017 (Jul. 2022).
24. Neelakantan, P., Gangappa, M., Rajasekar, M., Sunil Kumar, T. & Suresh Reddy, G. Resource allocation for content distribution in IoT edge cloud computing environments using deep reinforcement learning. *J. High Speed Netw.* **30** (3), 409–426. https://doi.org/10.3233/JHS-230165 (2024).
25. Zhi, H. & Wang, Y. Network resource allocation method based on blockchain and federated learning in IoT., *J. Commun. Netw.* **26** (2), 225–238. https://doi.org/10.23919/JCN.2024.000007 (2024).
26. Zhao, L., Wang, J., Liu, J. & Kato, N. Optimal edge resource allocation in iot-based smart cities. *IEEE Netw.* **33** (2), 30–35. https://doi.org/10.1109/MNET.2019.1800221 (2019).
27. Sindhura, S. Resource allocation based on predictive load balancing approach in multi cloud environment. *SSRN Electron. J.* https://doi.org/10.2139/ssrn.3690113 (2020).
28. Aghapour, Z., Sharifian, S. & Taheri, H. Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments. *Comput. Netw.* **223**, 109577. https://doi.org/10.1016/j.comnet.2023.109577 (Mar. 2023).

Author contributions

Nahideh Derakhshanfard: Validation, Writing – original draft, Editing and Supervision. Lida Hosseinzadeh made substantial contributions to the conceptualization, data interpretation, and writing of the manuscript. Fahimeh Rashidjafari: Conceptualization, Editing.

Funding

This research received no specific grant from any funding agency, commercial, or not-for-profit sectors.

Declarations

Competing interests
The authors declare no competing interests.

Ethics approval
This study did not involve human participants or animals. Therefore, ethics approval was not applicable.

Consent to publish
All authors have read and approved the final version of this manuscript and consent to its publication.

Additional information
Correspondence and requests for materials should be addressed to N.D.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025